SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Fundamentals of Data Science **COURSE CODE: DSA04**

COURSE OUTCOME COVERED

CO1: *Analyse the role of data science, facets of data, and the big data ecosystem for informed decision-making. (BL2)*

QUESTIONS: 05 **Total Marks:100**

Annexure A

Exploratory Data Analysis - Based Practical Assessment

Code of Conduct:

I AKHILESH RAJESH GOVARE (192524055) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Exploratory Data Analysis-Based Practical Assessment

1. Education Analytics Dataset

Question 1

A college wants to analyze student performance based on attendance, study hours, internal marks, and final result. Perform EDA to identify the factors affecting student results.

Student_ID	Attendance	Study_Hours	Internal_Marks	Result
S01	92	5	45	Pass
S02	55	2	24	Fail
S03	78	4	38	Pass
S04	60	3	28	Fail
S05	88	5	42	Pass
S06	47	1	20	Fail
S07	95	6	48	Pass
S08	72	3	35	Pass
S09	50	2	22	Fail
S10	83	4	40	Pass
S11	66	3	30	Fail
S12	90	5	44	Pass
S13	58	2	25	Fail
S14	76	4	36	Pass
S15	62	3	29	Fail
S16	85	5	41	Pass
S17	45	1	18	Fail
S18	80	4	39	Pass
S19	68	3	32	Pass
S20	53	2	23	Fail

 # STEP 1: Install/Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")
plt.rcParams['figure.figsize'] = (8,5)
```

 # STEP 2: Upload the dataset (Colab only)

```
from google.colab import files
uploaded = files.upload()   # choose student_data_set__csv.xlsx

# Read the file (works whether it's .xlsx or .csv)
filename = list(uploaded.keys())[0]
if filename.endswith('.csv'):
    df = pd.read_csv(filename)
else:
    df = pd.read_excel(filename)

df.head()


# STEP 3: Basic Understanding of Data

print("Shape of dataset:", df.shape)
print("\nColumn Info:")
df.info()

print("\nStatistical Summary:")
display(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nResult Value Counts:")
print(df['Result'].value_counts())


# STEP 4: Univariate Analysis


# Distribution of Attendance
plt.figure()
sns.histplot(df['Attendance'], kde=True, bins=10, color='steelblue')
plt.title('Distribution of Attendance')
plt.show()

# Distribution of Study Hours
plt.figure()
sns.histplot(df['Study_Hours'], kde=True, bins=8, color='seagreen')
plt.title('Distribution of Study Hours')
plt.show()
```

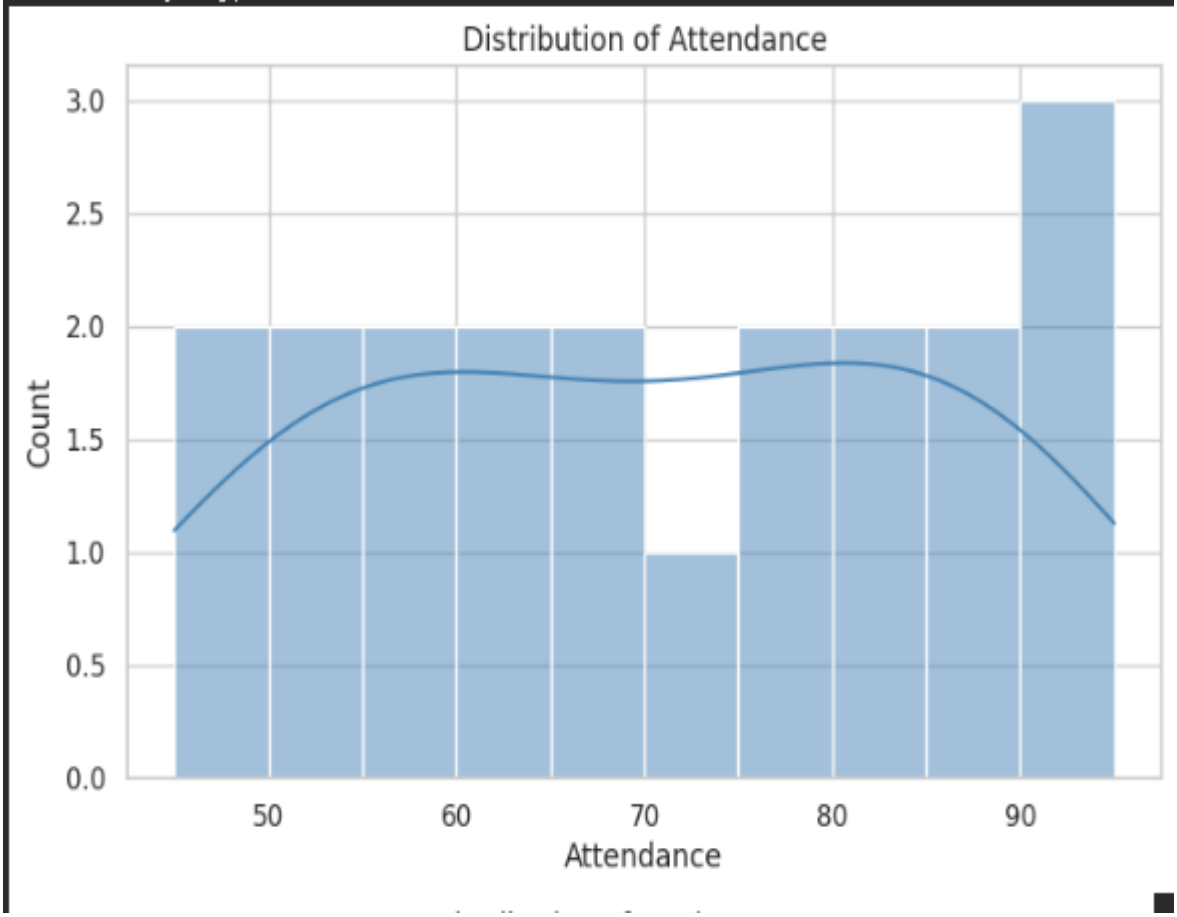
```
# Distribution of Internal Marks
plt.figure()
sns.histplot(df['Internal_Marks'], kde=True, bins=10, color='indianred')
plt.title('Distribution of Internal Marks')
plt.show()

# Pass/Fail count
plt.figure()
sns.countplot(x='Result', data=df, palette='Set2')
plt.title('Pass vs Fail Count')
plt.show()


# STEP 5: Bivariate Analysis (Feature vs Result)


# Attendance vs Result
plt.figure()
sns.boxplot(x='Result', y='Attendance', data=df, palette='pastel')
plt.title('Attendance by Result')
plt.show()

# Study Hours vs Result
plt.figure()
sns.boxplot(x='Result', y='Study_Hours', data=df, palette='pastel')
plt.title('Study Hours by Result')
plt.show()

# Internal Marks vs Result
plt.figure()
sns.boxplot(x='Result', y='Internal_Marks', data=df, palette='pastel')
plt.title('Internal Marks by Result')
plt.show()


# STEP 6: Scatter Plots (Relationships between features)

plt.figure()
sns.scatterplot(x='Attendance', y='Internal_Marks', hue='Result', data=df, s=100,
palette='deep')
plt.title('Attendance vs Internal Marks (colored by Result)')
plt.show()

plt.figure()
```

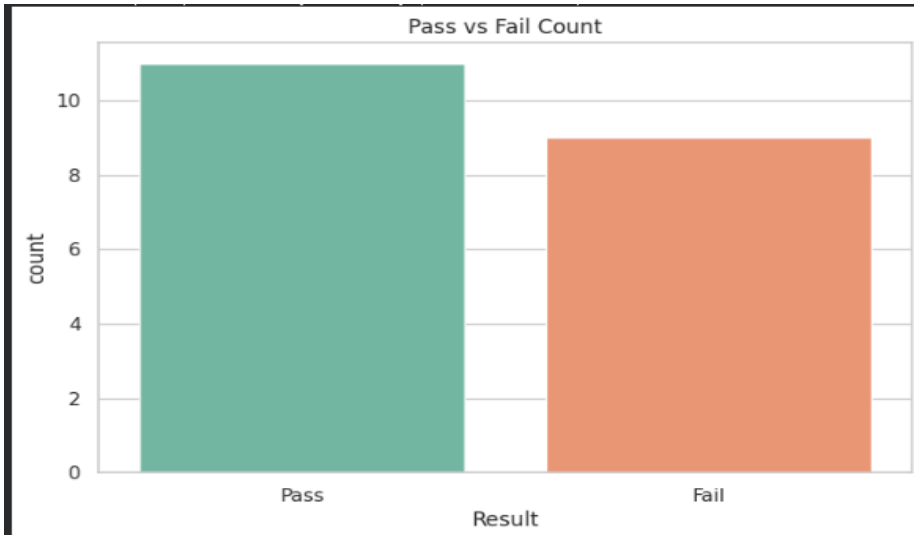
```
sns.scatterplot(x='Study_Hours', y='Internal_Marks', hue='Result', data=df, s=100, palette='deep')
plt.title('Study Hours vs Internal Marks (colored by Result)')
plt.show()


# STEP 7: Correlation Analysis


# Correlation matrix (numeric columns only)
numeric_df = df[['Attendance', 'Study_Hours', 'Internal_Marks']]
corr = numeric_df.corr()

plt.figure()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap')
plt.show()

print("Correlation with Internal Marks:")
print(corr['Internal_Marks'].sort_values(ascending=False))


# STEP 8: Pairplot (overall relationships)

sns.pairplot(df, vars=['Attendance', 'Study_Hours', 'Internal_Marks'], hue='Result', palette='husl')
plt.suptitle('Pairplot of Features by Result', y=1.02)
plt.show()


# STEP 9: Group-wise Mean Comparison

print("Average values grouped by Result:")
display(df.groupby('Result')[['Attendance', 'Study_Hours', 'Internal_Marks']].mean())


means = df.groupby('Result')[['Attendance', 'Study_Hours', 'Internal_Marks']].mean()
print("\n--- KEY INSIGHTS ---")
print(f"Average Attendance - Pass: {means.loc['Pass','Attendance']:.1f}%, Fail: {means.loc['Fail','Attendance']:.1f}%")
print(f"Average Study Hours - Pass: {means.loc['Pass','Study_Hours']:.1f} hrs, Fail: {means.loc['Fail','Study_Hours']:.1f} hrs")
print(f"Average Internal Marks - Pass: {means.loc['Pass','Internal_Marks']:.1f}, Fail: {means.loc['Fail','Internal_Marks']:.1f}")
```

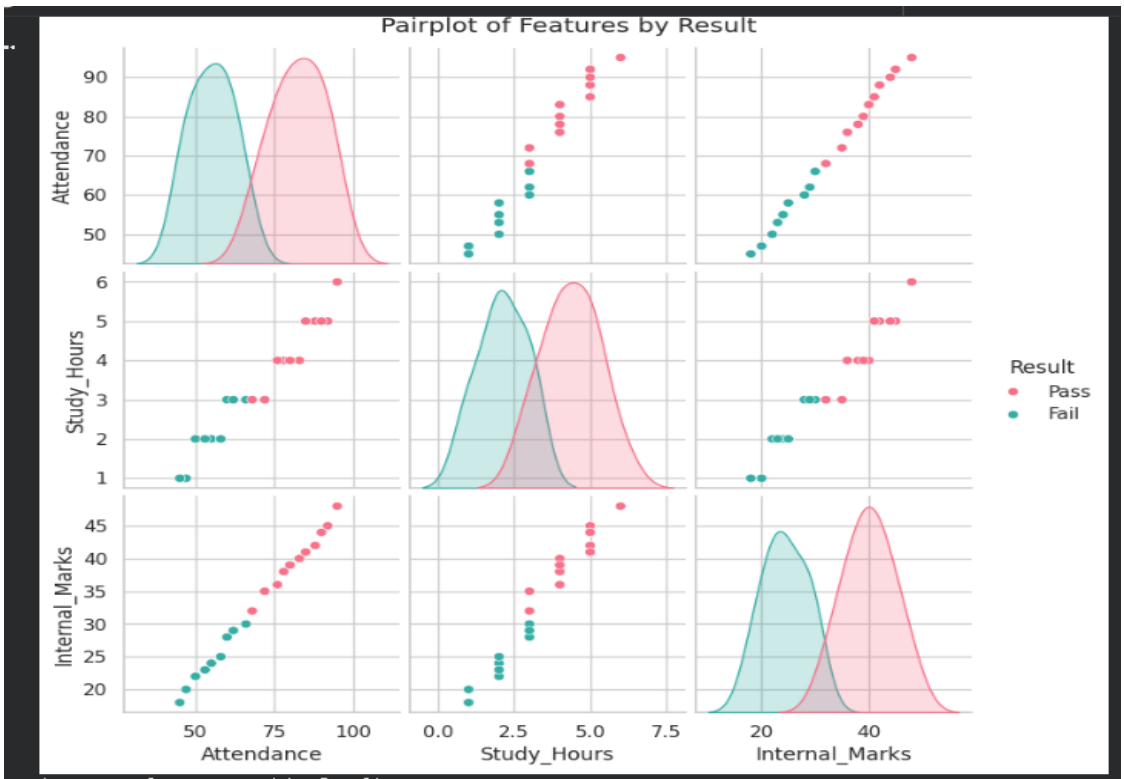
OUTPUT:

25%	57.250000	2.000000	24.750000
50%	70.000000	3.000000	33.500000
75%	83.500000	4.250000	40.250000
max	95.000000	6.000000	48.000000

```
Missing Values:
Student_ID        0
Attendance        0
Study_Hours       0
Internal_Marks    0
Result            0
dtype: int64

Result Value Counts:
Result
Pass    11
Fail     9
Name: count, dtype: int64
```







Average values grouped by Result:

Result	Attendance	Study_Hours	Internal_Marks
Fail	55.111111	2.111111	24.333333
Pass	82.454545	4.363636	40.000000

2. Healthcare Analytics Dataset

Question 2

A hospital wants to analyze patient health risk based on age, blood sugar, blood pressure, BMI, and disease status. Perform EDA to find important health patterns.

Patient_ID	Age	Sugar_Level	BP	BMI	Disease_Status
P01	45	145	130	28	Yes
P02	32	98	118	23	No
P03	55	180	145	31	Yes
P04	28	90	110	22	No
P05	60	200	150	34	Yes
P06	40	120	125	26	No
P07	52	165	140	30	Yes
P08	35	105	120	24	No
P09	48	155	135	29	Yes
P10	30	95	115	23	No
P11	62	210	155	35	Yes
P12	38	115	122	25	No
P13	50	170	142	30	Yes
P14	27	88	108	21	No
P15	58	190	148	33	Yes
P16	42	130	128	27	No
P17	46	150	132	28	Yes
P18	34	100	116	24	No
P19	53	175	144	31	Yes
P20	29	92	112	22	No

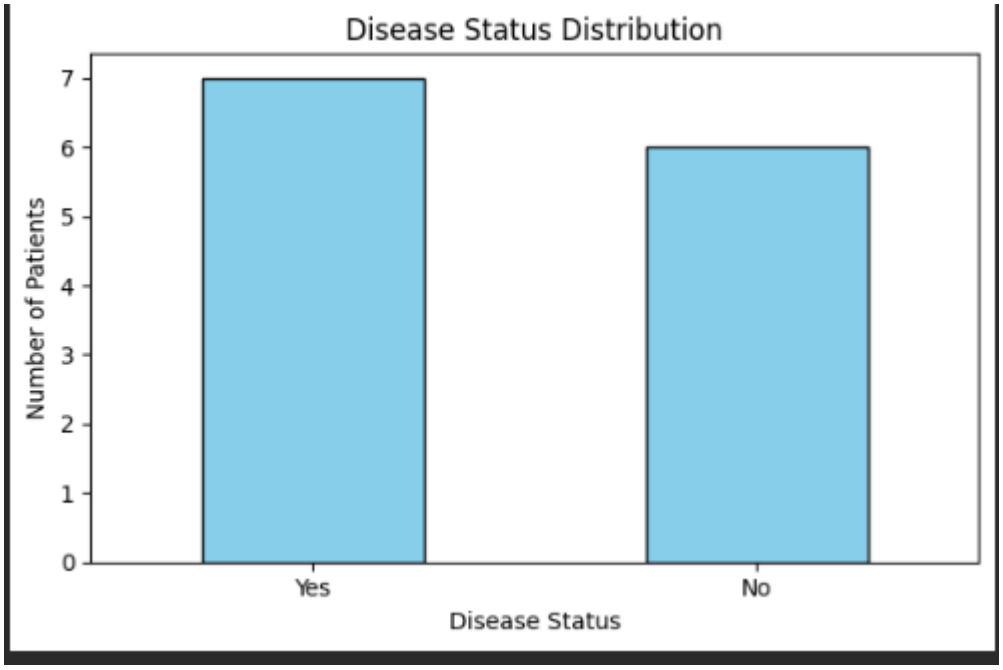
```
# Import Libraries
import pandas as pd
import matplotlib.pyplot as plt

# Load Dataset
df = pd.read_excel("hospital management.xlsx")
# Display First 5 Records

print("First 5 Records")
print(df.head())

# Dataset Information
print("\nDataset Information")
print(df.info())
# Check Missing Values
print("\nMissing Values")
print(df.isnull().sum())

# Statistical Summary
print("\nStatistical Summary")
print(df.describe())

# Check Duplicate Records
```

```
print("\nDuplicate Records:", df.duplicated().sum())

# Disease Status Count (Bar Graph)

print("\nDisease Status Distribution")
print(df['Disease_Status'].value_counts())

plt.figure(figsize=(6, 4))
df['Disease_Status'].value_counts().plot(kind='bar', color='skyblue', edgecolor='black')
plt.title("Disease Status Distribution")
plt.xlabel("Disease Status")
plt.ylabel("Number of Patients")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# Average Values by Disease Status (Bar Graph)

print("\nAverage Health Parameters by Disease Status")

group = df.groupby('Disease_Status')[['Age', 'Blood_Sugar', 'Blood_Pressure', 'BMI']].mean()
print(group)

group.plot(kind='bar', figsize=(8, 5), edgecolor='black')
plt.title("Average Health Parameters by Disease Status")
plt.xlabel("Disease Status")
plt.ylabel("Average Value")
plt.xticks(rotation=0)
plt.legend(title="Parameter")
plt.tight_layout()
plt.show()


# Age Group Distribution (Bar Graph)

print("\nAge Group Distribution")

bins = [0, 20, 30, 40, 50, 60, 100]
labels = ['0-20', '21-30', '31-40', '41-50', '51-60', '60+']
df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels)

age_group_counts = df['Age_Group'].value_counts().sort_index()
print(age_group_counts)

plt.figure(figsize=(6, 4))
age_group_counts.plot(kind='bar', color='lightgreen', edgecolor='black')
plt.title("Age Group Distribution")
plt.xlabel("Age Group")
plt.ylabel("Number of Patients")
```

```
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()


# Correlation Matrix (still useful context, printed only)

correlation = df[['Age', 'Blood_Sugar', 'Blood_Pressure', 'BMI']].corr()
print("\nCorrelation Matrix")
print(correlation)

print("\nEDA Completed Successfully!")
```

OUTPUT :

```
First 5 Records
  Patient_ID  Age  Sugar_Level   BP  BMI Disease_Status
0        P01   45          145  130   28            Yes
1        P02   32           98  118   23             No
2        P03   55          180  145   31            Yes
3        P04   28           90  110   22             No
4        P05   60          200  150   34            Yes

Dataset Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13 entries, 0 to 12
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Patient_ID      13 non-null     object
 1   Age             13 non-null     int64
 2   Sugar_Level     13 non-null     int64
 3   BP              13 non-null     int64
 4   BMI             13 non-null     int64
 5   Disease_Status  13 non-null     object
dtypes: int64(4), object(2)
memory usage: 756.0+ bytes
None

Missing Values
Patient_ID        0
Age               0
Sugar_Level       0
BP                0
BMI               0
Disease_Status    0
dtype: int64

Statistical Summary
             Age  Sugar_Level          BP        BMI
count  13.000000    13.000000   13.000000  13.000000
mean   44.230769   142.153846  131.307692  27.692308
std    11.351607    41.184233   14.302591   4.250189
min    28.000000    90.000000  110.000000  22.000000
25%    35.000000   105.000000  120.000000  24.000000
50%    45.000000   145.000000  130.000000  28.000000
75%    52.000000   170.000000  142.000000  30.000000
max    62.000000   210.000000  155.000000  35.000000

Duplicate Records: 0

Disease Status Distribution
Disease_Status
```


Disease Status Distribution
Number of Patients
0
1
2
3
4
5
6
7
Yes
No
Disease Status

3. Retail Sales Dataset
Question 3
A supermarket wants to analyze product sales based on category, price, quantity sold, discount, and revenue. Perform EDA to identify high-selling products and revenue patterns.

Product_ID	Category	Price	Quantity_Sold	Discount	Revenue
PR01	Grocery	50	20	5	1000
PR02	Snacks	30	35	3	1050
PR03	Dairy	45	18	2	810
PR04	Fruits	60	25	5	1500
PR05	Vegetables	40	30	4	1200
PR06	Grocery	80	15	6	1200
PR07	Snacks	25	40	3	1000
PR08	Dairy	55	16	2	880
PR09	Fruits	70	22	5	1540
PR10	Vegetables	35	28	4	980
PR11	Grocery	90	12	6	1080
PR12	Snacks	20	45	2	900
PR13	Dairy	65	14	3	910
PR14	Fruits	75	20	5	1500
PR15	Vegetables	45	26	4	1170
PR16	Grocery	100	10	7	1000
PR17	Snacks	35	32	3	1120
PR18	Dairy	50	19	2	950
PR19	Fruits	80	18	6	1440
PR20	Vegetables	30	35	3	1050

```
# Retail Sales Dataset - EDA
# Import Libraries
import pandas as pd
import matplotlib.pyplot as plt

# Load Dataset
df = pd.read_excel("grocery management.xlsx")

# Display First 5 Records
print("First 5 Records")
print(df.head())

# Dataset Information

print("\nDataset Information")
print(df.info())


# Missing Values
print("\nMissing Values")
print(df.isnull().sum())
```

```
# Statistical Summary

print("\nStatistical Summary")
print(df.describe())


# Duplicate Records

print("\nDuplicate Records:", df.duplicated().sum())


# Category Distribution

print("\nCategory Count")
print(df["Category"].value_counts())

plt.figure(figsize=(8,5))
df["Category"].value_counts().plot(kind="bar")
plt.title("Products by Category")
plt.xlabel("Category")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()


# Price Distribution

plt.figure(figsize=(6,4))
plt.hist(df["Price"], bins=10)
plt.title("Price Distribution")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.show()

# ------------------------------------
# Quantity Sold Distribution
# ------------------------------------
plt.figure(figsize=(6,4))
plt.hist(df["Quantity_Sold"], bins=10)
plt.title("Quantity Sold Distribution")
plt.xlabel("Quantity Sold")
plt.ylabel("Frequency")
plt.show()


# Discount Distribution

plt.figure(figsize=(6,4))
```

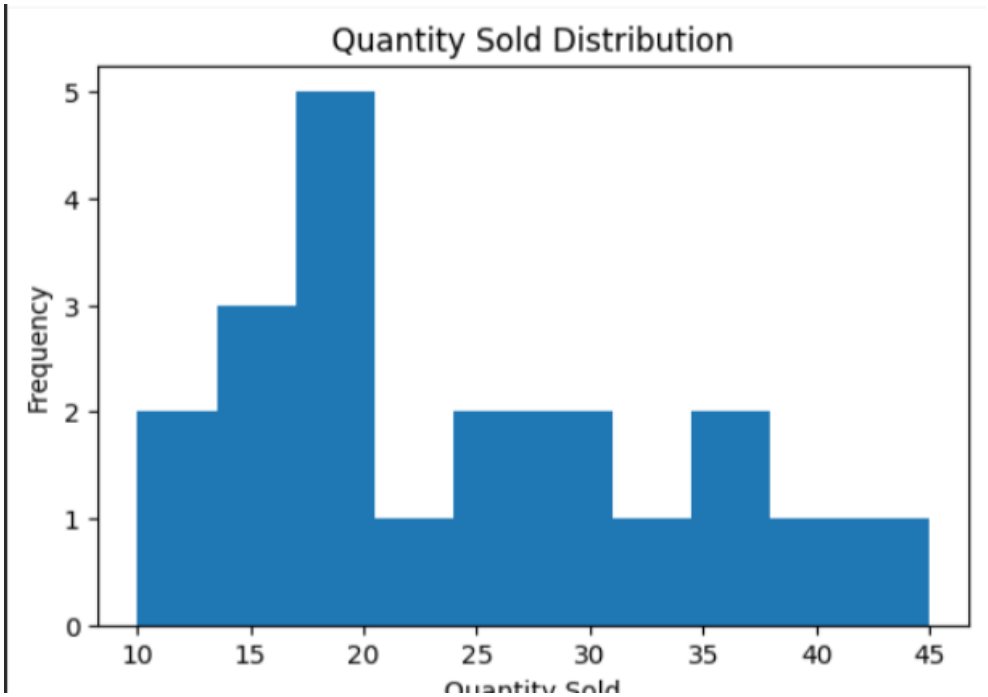
```
plt.hist(df["Discount"], bins=10)
plt.title("Discount Distribution")
plt.xlabel("Discount")
plt.ylabel("Frequency")
plt.show()


# Revenue Distribution

plt.figure(figsize=(6,4))
plt.hist(df["Revenue"], bins=10)
plt.title("Revenue Distribution")
plt.xlabel("Revenue")
plt.ylabel("Frequency")
plt.show()

print("\nAverage Sales by Category")

category_sales = df.groupby("Category")[["Quantity_Sold","Revenue"]].mean()

print(category_sales)

category_sales.plot(kind="bar", figsize=(8,5))
plt.title("Average Quantity Sold and Revenue by Category")
plt.ylabel("Average Value")
plt.xticks(rotation=45)
plt.show()

revenue = df.groupby("Category")["Revenue"].sum().sort_values(ascending=False)

print("\nTotal Revenue by Category")
print(revenue)

plt.figure(figsize=(8,5))
revenue.plot(kind="bar")
plt.title("Total Revenue by Category")
plt.xlabel("Category")
plt.ylabel("Revenue")
plt.xticks(rotation=45)
plt.show()

if "Product_Name" in df.columns:

    top_products                                                                    =
df.groupby("Product_Name")["Quantity_Sold"].sum().sort_values(ascending=False).hea
d(10)

    print("\nTop 10 High Selling Products")
    print(top_products)
```

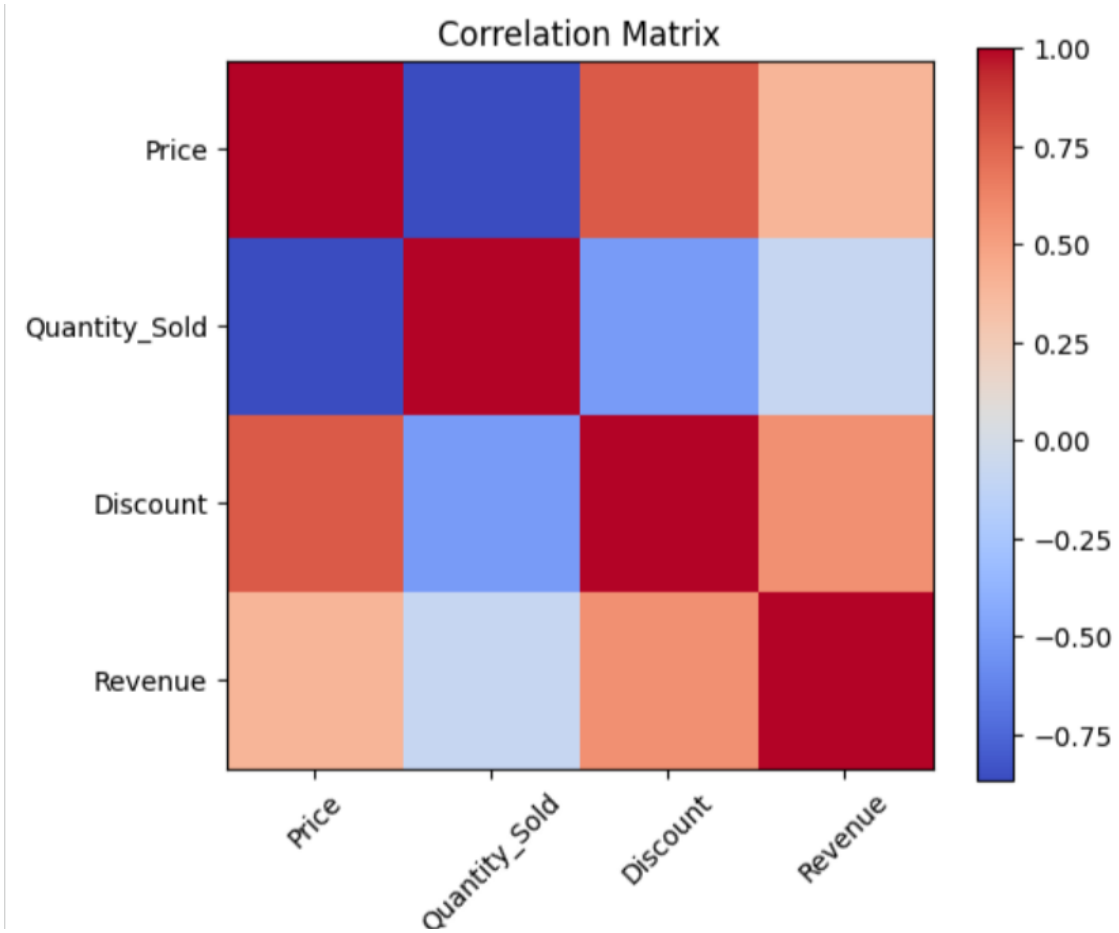
```
    plt.figure(figsize=(10,5))
    top_products.plot(kind="bar")
    plt.title("Top 10 High Selling Products")
    plt.xlabel("Product")
    plt.ylabel("Quantity Sold")
    plt.xticks(rotation=45)
    plt.show()

# Correlation Matrix
corr = df[["Price","Quantity_Sold","Discount","Revenue"]].corr()

print("\nCorrelation Matrix")
print(corr)

plt.figure(figsize=(6,5))
plt.imshow(corr, cmap="coolwarm")
plt.colorbar()

plt.xticks(range(len(corr.columns)), corr.columns, rotation=45)
plt.yticks(range(len(corr.columns)), corr.columns)

plt.title("Correlation Matrix")
plt.show()


plt.figure(figsize=(6,4))
plt.scatter(df["Price"], df["Revenue"])
plt.xlabel("Price")
plt.ylabel("Revenue")
plt.title("Price vs Revenue")
plt.show()

plt.figure(figsize=(6,4))
plt.scatter(df["Discount"], df["Revenue"])
plt.xlabel("Discount")
plt.ylabel("Revenue")
plt.title("Discount vs Revenue")
plt.show()


# Boxplots
plt.figure(figsize=(6,4))
plt.boxplot(df["Price"])
plt.title("Price Boxplot")
plt.show()

plt.figure(figsize=(6,4))
plt.boxplot(df["Quantity_Sold"])
plt.title("Quantity Sold Boxplot")
plt.show()
```

```
plt.figure(figsize=(6,4))
plt.boxplot(df["Discount"])
plt.title("Discount Boxplot")
plt.show()

plt.figure(figsize=(6,4))
plt.boxplot(df["Revenue"])
plt.title("Revenue Boxplot")
plt.show()

print("\nEDA Completed Successfully!")
```

OUTPUT :

```
First 5 Records
  Product_ID    Category  Price  Quantity_Sold  Discount  Revenue
0       PR01     Grocery     50             20         5     1000
1       PR02      Snacks     30             35         3     1050
2       PR03       Dairy     45             18         2      810
3       PR04      Fruits     60             25         5     1500
4       PR05  Vegetables     40             30         4     1200

Dataset Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Product_ID     20 non-null     object
 1   Category       20 non-null     object
 2   Price          20 non-null     int64
 3   Quantity_Sold  20 non-null     int64
 4   Discount       20 non-null     int64
 5   Revenue        20 non-null     int64
dtypes: int64(4), object(2)
memory usage: 1.1+ KB
None

Missing Values
Product_ID       0
Category         0
Price            0
Quantity_Sold    0
Discount         0
Revenue          0
dtype: int64

Statistical Summary
            Price  Quantity_Sold   Discount      Revenue
count   20.000000      20.000000  20.000000    20.000000
mean    54.000000      24.000000   4.000000  1114.000000
std     22.803509       9.673621   1.555973   221.416493
min     20.000000      10.000000   2.000000   810.000000
25%     35.000000      17.500000   3.000000   972.500000
50%     50.000000      21.000000   4.000000  1050.000000
75%     71.250000      30.500000   5.000000  1200.000000
max    100.000000      45.000000   7.000000  1540.000000

Duplicate Records: 0

Category Count
Category
Grocery       4
Snacks        4
Dairy         4
Fruits        4
Vegetables    4
Name: count, dtype: int64
```


Quantity Sold Distribution
Frequency
0
1
2
3
4
5
10
15
20
25
30
35
40
45


Correlation Matrix
Price
Quantity_Sold
Discount
Revenue
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75

4. Banking Loan Approval Dataset

Question 4

A bank wants to analyze loan approval decisions based on income, credit score, loan amount, employment type, and approval status. Perform EDA to identify patterns related to loan approval.

Customer_ID	Income	Credit_Score	Loan_Amount	Employment_Type	Loan_Status
C01	45000	720	200000	Salaried	Approved
C02	25000	580	150000	Self-employed	Rejected
C03	60000	750	300000	Salaried	Approved
C04	22000	560	120000	Unemployed	Rejected
C05	50000	710	250000	Salaried	Approved
C06	30000	600	180000	Self-employed	Rejected
C07	70000	780	350000	Salaried	Approved
C08	28000	590	160000	Self-employed	Rejected
C09	55000	730	270000	Salaried	Approved
C10	24000	570	130000	Unemployed	Rejected
C11	65000	760	320000	Salaried	Approved
C12	35000	620	200000	Self-employed	Rejected
C13	48000	700	240000	Salaried	Approved
C14	26000	585	140000	Unemployed	Rejected
C15	75000	790	400000	Salaried	Approved
C16	32000	610	190000	Self-employed	Rejected
C17	52000	725	260000	Salaried	Approved
C18	27000	575	150000	Self-employed	Rejected
C19	68000	770	330000	Salaried	Approved
C20	23000	555	125000	Unemployed	Rejected

```
# Banking Loan Approval Dataset - EDA
# Import Libraries
import pandas as pd
import matplotlib.pyplot as plt

# Load Dataset
df = pd.read_excel("bank management.xlsx")

# Display First 5 Records

print("First 5 Records")
print(df.head())


# Dataset Information
print("\nDataset Information")
print(df.info())

# Check Missing Values

print("\nMissing Values")
print(df.isnull().sum())
```

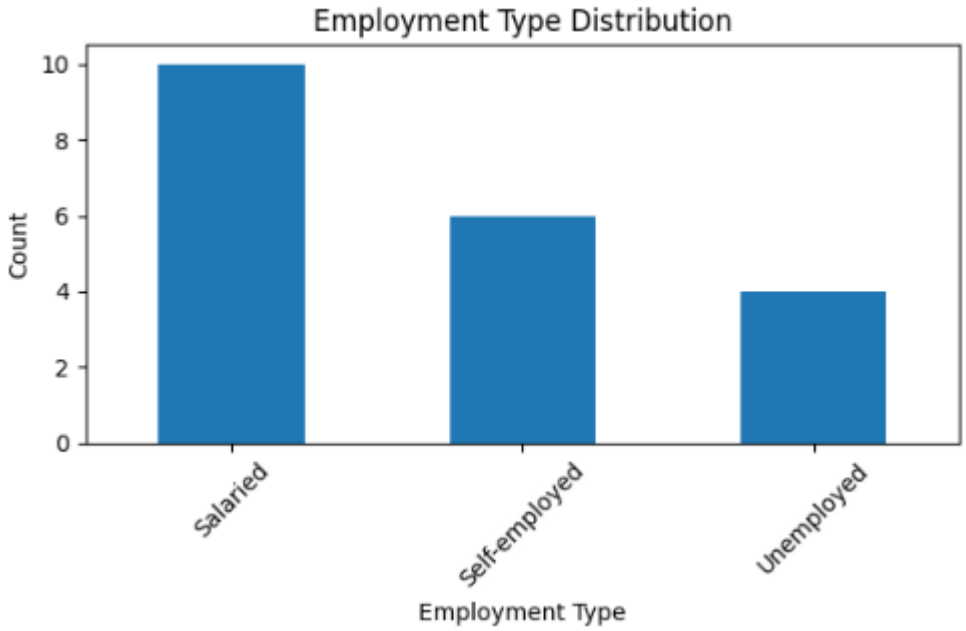
```
# Statistical Summary

print("\nStatistical Summary")
print(df.describe())


# Check Duplicate Records

print("\nDuplicate Records:", df.duplicated().sum())


# Approval Status Distribution (Bar Chart)

print("\nApproval Status Count")
print(df["Loan_Status"].value_counts())

plt.figure(figsize=(6,4))
df["Loan_Status"].value_counts().plot(kind="bar")
plt.title("Loan Approval Status")
plt.xlabel("Approval Status")
plt.ylabel("Number of Applications")
plt.tight_layout()
plt.show()

# Employment Type Distribution (Bar Chart)
print("\nEmployment Type Count")
print(df["Employment_Type"].value_counts())

plt.figure(figsize=(6,4))
df["Employment_Type"].value_counts().plot(kind="bar")
plt.title("Employment Type Distribution")
plt.xlabel("Employment Type")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()


# Income Distribution (Bar Chart using bins)
income_bins = pd.cut(df["Income"], bins=10)
income_counts = income_bins.value_counts().sort_index()

print("\nIncome Distribution (Binned)")
print(income_counts)

plt.figure(figsize=(7,4))
income_counts.plot(kind="bar")
plt.title("Income Distribution")
plt.xlabel("Income Range")
```

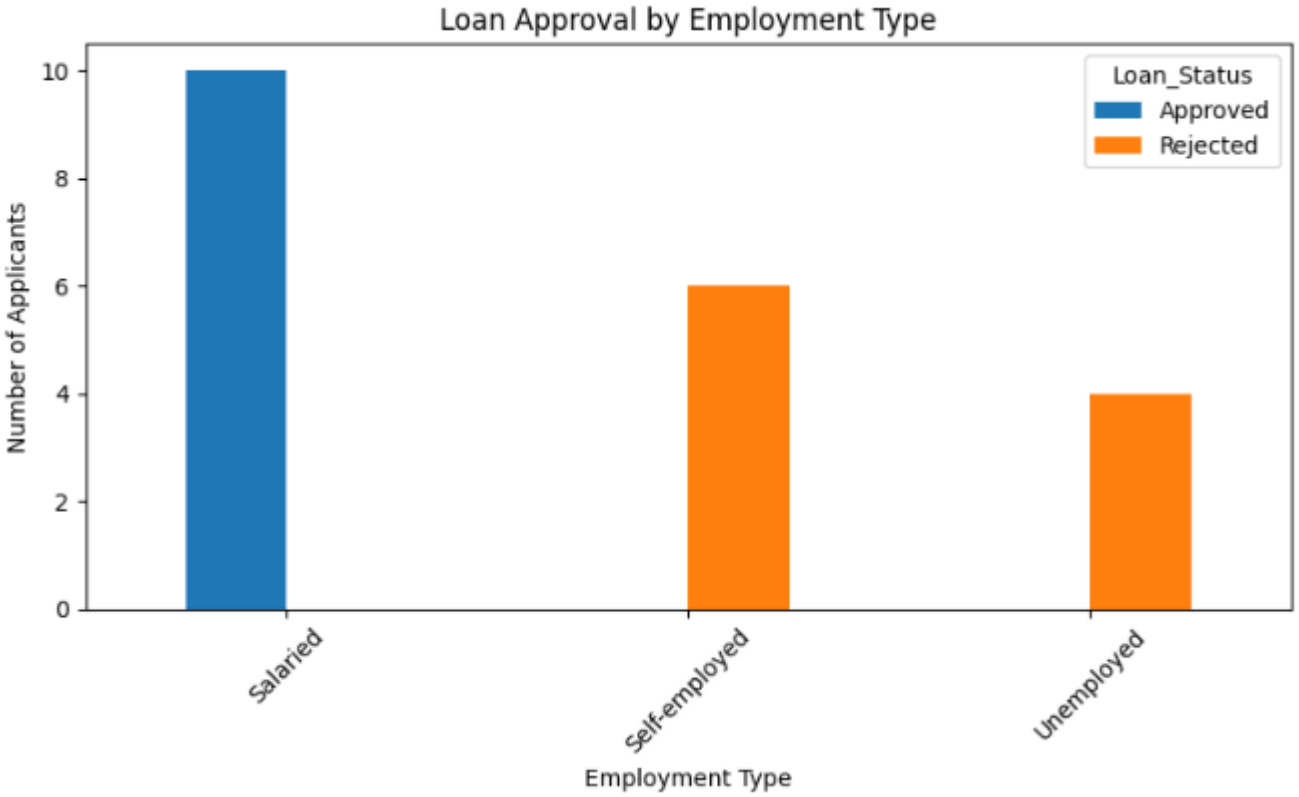
```
plt.ylabel("Frequency")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()


# Credit Score Distribution (Bar Chart using bins)
credit_bins = pd.cut(df["Credit_Score"], bins=10)
credit_counts = credit_bins.value_counts().sort_index()

print("\nCredit Score Distribution (Binned)")
print(credit_counts)

plt.figure(figsize=(7,4))
credit_counts.plot(kind="bar")
plt.title("Credit Score Distribution")
plt.xlabel("Credit Score Range")
plt.ylabel("Frequency")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

# Loan Amount Distribution (Bar Chart using bins)

loan_bins = pd.cut(df["Loan_Amount"], bins=10)
loan_counts = loan_bins.value_counts().sort_index()

print("\nLoan Amount Distribution (Binned)")
print(loan_counts)

plt.figure(figsize=(7,4))
loan_counts.plot(kind="bar")
plt.title("Loan Amount Distribution")
plt.xlabel("Loan Amount Range")
plt.ylabel("Frequency")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()


# Average Values by Approval Status (Bar Chart)

approval                                                                    =
df.groupby("Loan_Status")[["Income","Credit_Score","Loan_Amount"]].mean()

print("\nAverage Values by Approval Status")
print(approval)

approval.plot(kind="bar", figsize=(8,5))
plt.title("Average Income, Credit Score and Loan Amount")
```

```
plt.ylabel("Average Value")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()


# Loan Approval by Employment Type (Bar Chart)
employment = pd.crosstab(df["Employment_Type"], df["Loan_Status"])

print("\nLoan Approval by Employment Type")
print(employment)

employment.plot(kind="bar", figsize=(8,5))
plt.title("Loan Approval by Employment Type")
plt.xlabel("Employment Type")
plt.ylabel("Number of Applicants")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()


# Correlation Summary (Bar Chart instead of heatmap)

corr = df[["Income","Credit_Score","Loan_Amount"]].corr()

print("\nCorrelation Matrix")
print(corr)

# Show correlation of Loan_Amount with Income and Credit_Score as a bar chart
loan_corr = corr["Loan_Amount"].drop("Loan_Amount")

plt.figure(figsize=(6,4))
loan_corr.plot(kind="bar")
plt.title("Correlation of Loan Amount with Income & Credit Score")
plt.xlabel("Feature")
plt.ylabel("Correlation Coefficient")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

print("\nEDA Completed Successfully!")
```

OUTPUT :

```
First 5 Records
  Customer_ID  Income  Credit_Score  Loan_Amount Employment_Type Loan_Status
0         C01   45000           720       200000        Salaried    Approved
1         C02   25000           580       150000   Self-employed    Rejected
2         C03   60000           750       300000        Salaried    Approved
3         C04   22000           560       120000      Unemployed    Rejected
4         C05   50000           710       250000        Salaried    Approved

Dataset Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Customer_ID      20 non-null     object
 1   Income           20 non-null     int64
 2   Credit_Score     20 non-null     int64
 3   Loan_Amount      20 non-null     int64
 4   Employment_Type  20 non-null     object
 5   Loan_Status      20 non-null     object
dtypes: int64(3), object(3)
memory usage: 1.1+ KB
None

Missing Values
Customer_ID        0
Income             0
Credit_Score       0
Loan_Amount        0
Employment_Type    0
Loan_Status        0
dtype: int64

Statistical Summary
              Income  Credit_Score    Loan_Amount
count      20.000000     20.000000      20.000000
mean    43000.000000    664.000000  223250.000000
std     17923.815383     85.526235   83733.207272
min     22000.000000    555.000000  120000.000000
25%     26750.000000    583.750000  150000.000000
50%     40000.000000    660.000000  200000.000000
75%     56250.000000    735.000000  277500.000000
max     75000.000000    790.000000  400000.000000

Duplicate Records: 0

Approval Status Count
Loan_Status
Approved    10
Rejected    10
Name: count, dtype: int64
```


Employment Type Distribution
Count
10
8
6
4
2
0
Salaried
Self-employed
Unemployed
Employment Type


Loan Approval by Employment Type
Loan_Status
Approved
Rejected
Number of Applicants
10
8
6
4
2
0
Salaried
Self-employed
Unemployed
Employment Type

5. Agriculture Crop Yield Dataset
Question 5
An agricultural department wants to analyze crop yield based on rainfall, temperature, fertilizer usage, soil type, and crop yield. Perform EDA to identify the factors influencing crop production.

Farm_ID	Rainfall_mm	Temperature	Fertilizer_kg	Soil_Type	Crop_Yield_kg
F01	850	28	60	Loamy	3200
F02	600	32	45	Sandy	2100
F03	900	27	65	Loamy	3500
F04	500	34	40	Sandy	1800
F05	780	29	55	Clay	3000
F06	650	31	48	Sandy	2300
F07	920	26	70	Loamy	3700
F08	720	30	52	Clay	2800
F09	550	33	42	Sandy	1900
F10	800	28	58	Loamy	3100
F11	880	27	62	Loamy	3400
F12	620	32	46	Sandy	2200
F13	760	29	54	Clay	2950
F14	940	26	72	Loamy	3800
F15	580	33	44	Sandy	2000
F16	820	28	60	Clay	3150
F17	700	30	50	Clay	2700
F18	960	25	75	Loamy	3900
F19	540	34	41	Sandy	1850
F20	790	29	56	Clay	3050

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

# Make sure "agriculture_crop_yield.csv" is in the same folder as this
# script (or, in Google Colab, upload it using the snippet below).

# ---- If using Google Colab, uncomment these 2 lines to upload the file ----
# from google.colab import files
# uploaded = files.upload()   # choose agriculture_crop_yield.csv when prompted

df = pd.read_csv("agriculture_crop_yield.csv")

print("=" * 60)
print("STEP 1: DATASET OVERVIEW")
print("=" * 60)
print("\nFirst 5 rows:\n", df.head())
print("\nShape of dataset (rows, columns):", df.shape)
```

```
print("\nColumn names:", list(df.columns))


print("\n" + "=" * 60)
print("STEP 2: DATASET INFO")
print("=" * 60)
print(df.info())

print("\nMissing values in each column:\n", df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())



print("\n" + "=" * 60)
print("STEP 3: STATISTICAL SUMMARY (Numeric Columns)")
print("=" * 60)
print(df.describe())

print("\n" + "=" * 60)
print("STEP 3b: SOIL TYPE VALUE COUNTS")
print("=" * 60)
print(df["Soil_Type"].value_counts())



print("\n" + "=" * 60)
print("STEP 4: CORRELATION MATRIX (Numeric Columns)")
print("=" * 60)
numeric_cols = ["Rainfall_mm", "Temperature", "Fertilizer_kg", "Crop_Yield_kg"]
corr_matrix = df[numeric_cols].corr()
print(corr_matrix)

print("\nCorrelation of each factor with Crop_Yield_kg:")
print(corr_matrix["Crop_Yield_kg"].sort_values(ascending=False))



print("\n" + "=" * 60)
print("STEP 5: AVERAGE CROP YIELD BY SOIL TYPE")
print("=" * 60)
soil_yield_avg                                                                    =
df.groupby("Soil_Type")["Crop_Yield_kg"].mean().sort_values(ascending=False)
print(soil_yield_avg)
```

```
# ---- 6.1 BAR GRAPH: Average Crop Yield by Soil Type ----
plt.figure(figsize=(7, 5))
soil_yield_avg.plot(kind="bar", color=["#4C72B0", "#DD8452", "#55A868"])
plt.title("Average Crop Yield by Soil Type")
plt.xlabel("Soil Type")
plt.ylabel("Average Crop Yield (kg)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# ---- 6.2 BAR GRAPH: Crop Yield for every Farm ----
plt.figure(figsize=(10, 5))
plt.bar(df["Farm_ID"], df["Crop_Yield_kg"], color="seagreen")
plt.title("Crop Yield per Farm")
plt.xlabel("Farm ID")
plt.ylabel("Crop Yield (kg)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# ---- 6.3 BAR GRAPH: Correlation of each factor with Crop Yield ----
corr_with_yield = corr_matrix["Crop_Yield_kg"].drop("Crop_Yield_kg")
plt.figure(figsize=(7, 5))
corr_with_yield.plot(kind="bar", color="darkorange")
plt.title("Correlation of Factors with Crop Yield")
plt.xlabel("Factor")
plt.ylabel("Correlation Coefficient")
plt.axhline(0, color="black", linewidth=0.8)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# ---- 6.4 BAR GRAPH: Binned Rainfall vs Average Crop Yield ----
# pd.cut() is used to convert the continuous Rainfall_mm column
# into ranges (bins), then the average yield per bin is plotted.
rainfall_bins = pd.cut(df["Rainfall_mm"], bins=[500, 650, 800, 950],
                       labels=["500-650", "650-800", "800-950"])
rainfall_yield_avg = df.groupby(rainfall_bins)["Crop_Yield_kg"].mean()

plt.figure(figsize=(7, 5))
rainfall_yield_avg.plot(kind="bar", color="steelblue")
plt.title("Average Crop Yield by Rainfall Range")
```

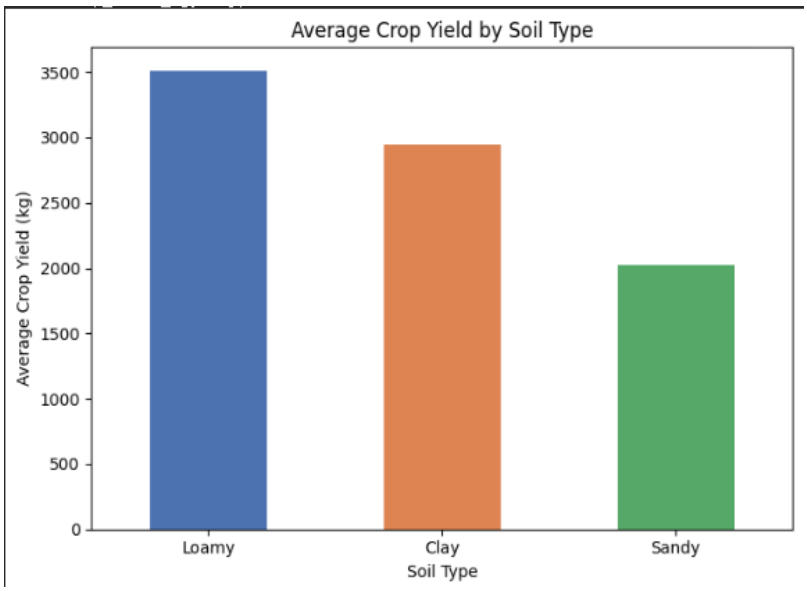
```
plt.xlabel("Rainfall Range (mm)")
plt.ylabel("Average Crop Yield (kg)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# ---- 6.5 BAR GRAPH: Binned Fertilizer Usage vs Average Crop Yield ----
fertilizer_bins = pd.cut(df["Fertilizer_kg"], bins=[40, 50, 60, 70, 80],
                         labels=["40-50", "50-60", "60-70", "70-80"])
fertilizer_yield_avg = df.groupby(fertilizer_bins)["Crop_Yield_kg"].mean()

plt.figure(figsize=(7, 5))
fertilizer_yield_avg.plot(kind="bar", color="mediumpurple")
plt.title("Average Crop Yield by Fertilizer Usage Range")
plt.xlabel("Fertilizer Usage Range (kg)")
plt.ylabel("Average Crop Yield (kg)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# ---- 6.6 LINE GRAPH: Rainfall vs Crop Yield trend ----
# Sorted by Rainfall so the line shows a clear increasing/decreasing trend.
df_sorted_rain = df.sort_values("Rainfall_mm")
plt.figure(figsize=(8, 5))
plt.plot(df_sorted_rain["Rainfall_mm"], df_sorted_rain["Crop_Yield_kg"],
        marker="o", color="royalblue")
plt.title("Trend: Rainfall vs Crop Yield")
plt.xlabel("Rainfall (mm)")
plt.ylabel("Crop Yield (kg)")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

# ---- 6.7 LINE GRAPH: Temperature vs Crop Yield trend ----
df_sorted_temp = df.sort_values("Temperature")
plt.figure(figsize=(8, 5))
plt.plot(df_sorted_temp["Temperature"], df_sorted_temp["Crop_Yield_kg"],
        marker="o", color="crimson")
plt.title("Trend: Temperature vs Crop Yield")
plt.xlabel("Temperature (°C)")
plt.ylabel("Crop Yield (kg)")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```

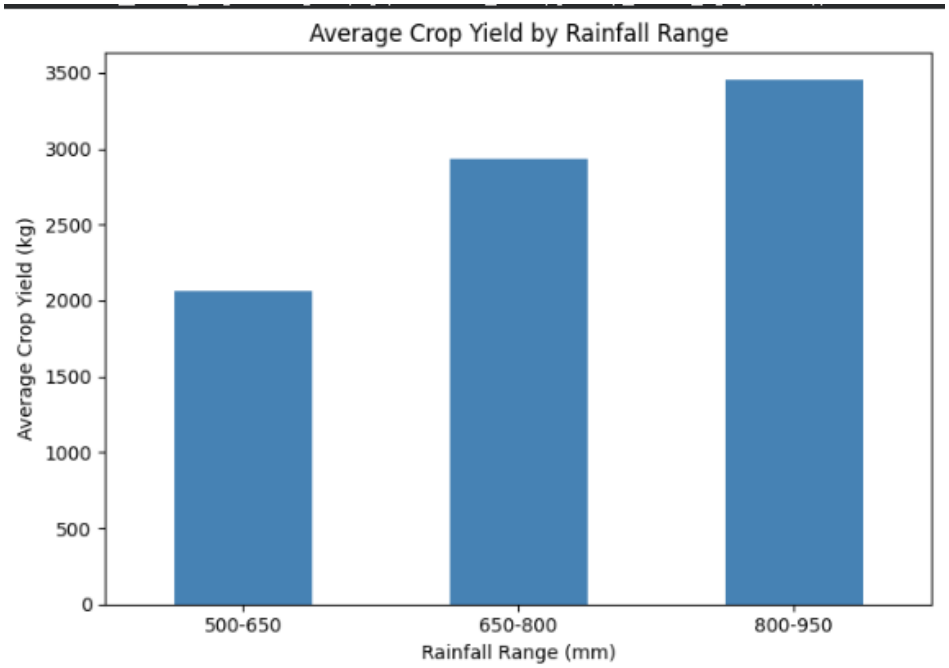
```
# ---- 6.8 LINE GRAPH: Fertilizer Usage vs Crop Yield trend ----
df_sorted_fert = df.sort_values("Fertilizer_kg")
plt.figure(figsize=(8, 5))
plt.plot(df_sorted_fert["Fertilizer_kg"], df_sorted_fert["Crop_Yield_kg"],
        marker="o", color="darkgreen")
plt.title("Trend: Fertilizer Usage vs Crop Yield")
plt.xlabel("Fertilizer Usage (kg)")
plt.ylabel("Crop Yield (kg)")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

# ---- 6.9 LINE GRAPH: Crop Yield across all farms (Farm_ID order) ----
plt.figure(figsize=(10, 5))
plt.plot(df["Farm_ID"], df["Crop_Yield_kg"], marker="o",
        linestyle="-", color="teal")
plt.title("Crop Yield Trend Across Farms (F01 to F20)")
plt.xlabel("Farm ID")
plt.ylabel("Crop Yield (kg)")
plt.xticks(rotation=45)
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()



print("\n" + "=" * 60)
print("STEP 7: KEY FINDINGS")
print("=" * 60)
strongest_factor = corr_with_yield.abs().idxmax()
print(f"1. The factor with the strongest correlation to Crop Yield is: "
    f"{strongest_factor} (correlation = {corr_with_yield[strongest_factor]:.2f})")
print(f"2. Soil type with the highest average yield: "
    f"{soil_yield_avg.idxmax()} ({soil_yield_avg.max():.1f} kg)")
print(f"3. Soil type with the lowest average yield: "
    f"{soil_yield_avg.idxmin()} ({soil_yield_avg.min():.1f} kg)")
print(f"4. Highest rainfall range yield: "
    f"{rainfall_yield_avg.idxmax()} mm -> {rainfall_yield_avg.max():.1f} kg")
print(f"5. Highest fertilizer usage range yield: "
    f"{fertilizer_yield_avg.idxmax()} kg -> {fertilizer_yield_avg.max():.1f} kg")
```

OUTPUT :

```
============================================================
STEP 1: DATASET OVERVIEW
============================================================

First 5 rows:
   Farm_ID  Rainfall_mm  Temperature  Fertilizer_kg Soil_Type  Crop_Yield_kg
0     F01          850           28             60     Loamy           3200
1     F02          600           32             45     Sandy           2100
2     F03          900           27             65     Loamy           3500
3     F04          500           34             40     Sandy           1800
4     F05          780           29             55      Clay           3000

Shape of dataset (rows, columns): (20, 6)

Column names: ['Farm_ID', 'Rainfall_mm', 'Temperature', 'Fertilizer_kg', 'Soil_Type', 'Crop_Yield_kg']

============================================================
STEP 2: DATASET INFO
============================================================
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Farm_ID        20 non-null     object
 1   Rainfall_mm    20 non-null     int64
 2   Temperature    20 non-null     int64
 3   Fertilizer_kg  20 non-null     int64
 4   Soil_Type      20 non-null     object
 5   Crop_Yield_kg  20 non-null     int64
dtypes: int64(4), object(2)
memory usage: 1.1+ KB
None

Missing values in each column:
 Farm_ID          0
Rainfall_mm      0
Temperature      0
Fertilizer_kg    0
Soil_Type        0
Crop_Yield_kg    0
dtype: int64

Duplicate rows: 0
```

```
============================================================
STEP 3: STATISTICAL SUMMARY (Numeric Columns)
============================================================
       Rainfall_mm  Temperature  Fertilizer_kg  Crop_Yield_kg
count    20.000000    20.000000      20.000000      20.000000
mean    743.000000    29.550000      54.750000    2820.000000
std     144.335136     2.762055      10.507516     681.407213
min     500.000000    25.000000      40.000000    1800.000000
25%     615.000000    27.750000      45.750000    2175.000000
50%     770.000000    29.000000      54.500000    2975.000000
75%     857.500000    32.000000      60.500000    3250.000000
max     960.000000    34.000000      75.000000    3900.000000

============================================================
STEP 3b: SOIL TYPE VALUE COUNTS
============================================================
Soil_Type
Loamy    7
Sandy    7
Clay     6
Name: count, dtype: int64

============================================================
STEP 4: CORRELATION MATRIX (Numeric Columns)
============================================================
              Rainfall_mm  Temperature  Fertilizer_kg  Crop_Yield_kg
Rainfall_mm      1.000000    -0.993192       0.980550       0.995521
Temperature     -0.993192     1.000000      -0.979735      -0.993300
Fertilizer_kg    0.980550    -0.979735       1.000000       0.983181
Crop_Yield_kg    0.995521    -0.993300       0.983181       1.000000

Correlation of each factor with Crop_Yield_kg:
Crop_Yield_kg    1.000000
Rainfall_mm      0.995521
Fertilizer_kg    0.983181
Temperature     -0.993300
Name: Crop_Yield_kg, dtype: float64

============================================================
STEP 5: AVERAGE CROP YIELD BY SOIL TYPE
============================================================
Soil_Type
Loamy    3514.285714
Clay     2941.666667
Sandy    2021.428571
Name: Crop_Yield_kg, dtype: float64
```




Average Crop Yield by Rainfall Range
Average Crop Yield (kg)
3500
3000
2500
2000
1500
1000
500
0
500-650
650-800
800-950
Rainfall Range (mm)

Common EDA Tasks:

1. Identify the number of rows and columns.
2. Check data types of each column.
3. Find missing values and duplicate values.
4. Calculate mean, median, minimum, maximum, and standard deviation.
5. Draw suitable charts such as bar chart, histogram, scatter plot, and box plot.
6. Identify important patterns from the dataset.
7. Write 4 to 5 observations based on EDA.
8. Give a short conclusion from the analysis.

RUBRICS FOR CALCULATION

Exploratory Data Analysis-Based Practical Assessment					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Dataset Understanding	20	Clearly explains dataset source, features, target variables, data types, and problem context with strong clarity	Explains dataset, features, and variables with adequate clarity	Basic explanation of dataset with limited understanding of variables	Poor understanding of dataset; variables and problem context are unclear
Data Cleaning	25	Effectively identifies and handles missing values, duplicates, incorrect entries, and outliers using suitable methods	Handles most missing values, duplicates, and errors correctly	Performs basic cleaning but misses some important data quality issues	Minimal or incorrect data cleaning; major errors remain in the dataset
Data Analysis and Statistical Summary	20	Provides detailed descriptive statistics, comparisons, patterns, and meaningful interpretations	Provides relevant summary statistics and basic interpretation	Provides limited statistics with weak explanation	Lacks statistical analysis or gives incorrect interpretation
Data Visualization	20	Uses highly appropriate charts with proper labels, titles, and clear visual interpretation	Uses suitable charts with mostly clear labels and explanation	Uses limited or basic charts with unclear interpretation	Poor or incorrect visualizations; charts do not support analysis
Insight Generation and Reporting	15	Presents strong insights, conclusions, and recommendations based on data evidence	Presents clear findings and conclusions with reasonable support	Provides basic findings but lacks depth or proper justification	Findings are unclear, unsupported, or not related to the data

CO1-ASSESSMENT TOOL -2 Concept Map

QUESTIONS : 5 **MARKS : 50**

Q No	Dataset Understanding (20%)	Data Cleaning (25%)	Data Analysis and Statistical Summary (20%)	Data Visualization (20%)	Insight Generation and Reporting (15%)	Total Score (100%)
1	/2.5	/2.5	/2.0	/2.0	/1.0	/10
2	/2.5	/2.5	/2.0	/2.0	/1.0	/10
3	/2.5	/2.5	/2.0	/2.0	/1.0	/10
4	/2.5	/2.5	/2.0	/2.0	/1.0	/10
5	/2.5	/2.5	/2.0	/2.0	/1.0	/10
Overall Rubrics Value	/12.5	/12.5	/10	/10	/10	/50
	/25	/25	/20	/20	/10	/100